



UITs

**UNIVERSITY OF INFORMATION
TECHNOLOGY AND SCIENCES**

Lab report

Course Title: Microprocessors and Microcontrollers Lab

Course Code: CSE-360

Submitted By

Name: Nafisa Sultana Juie

ID No: 0432320005101038

Section: 6A2

Department: CSE

Batch: 54

Semester: Spring-26

Submitted To

Md. Ismail

Lecturer, Department of CSE

Md. Faysal

Lecturer, Department of CSE

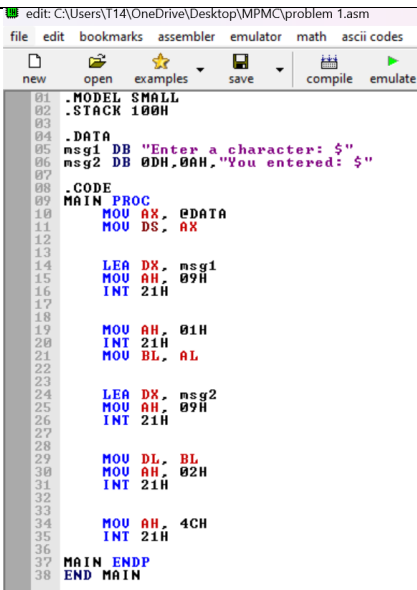
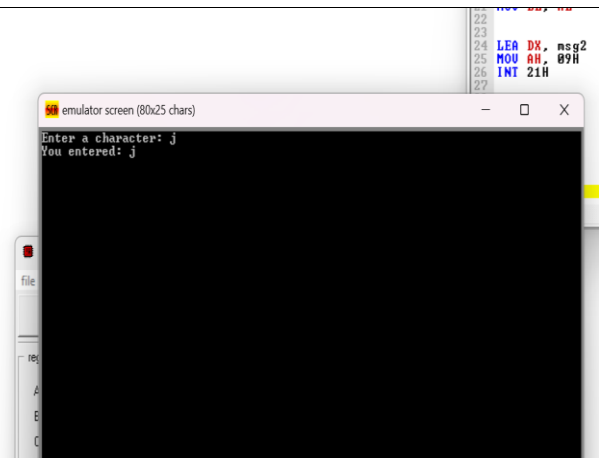
Date: 19/05/2026

Experiment No: 01

Experiment Name: Write an Assembly Language program to take input from the user and display the output.

Introduction:

Assembly Language is a low-level programming language that helps us understand how a computer works internally. It uses simple instructions that are directly related to machine language. In this experiment, an Assembly Language program was written to take input from the user and display the same output on the screen. The program uses DOS interrupt functions for input and output operations. This experiment helps to understand basic Assembly concepts such as registers, interrupts, and data transfer between registers.

Code	Output
 <pre>edit: C:\Users\T14\OneDrive\Desktop\MPMC\problem 1.asm file edit bookmarks assembler emulator math ascii codes new open examples save compile emulate 01 .MODEL SMALL 02 .STACK 100H 03 04 .DATA 05 msg1 DB "Enter a character: \$" 06 msg2 DB 0DH,0AH,"You entered: \$" 07 08 .CODE 09 MAIN PROC 10 MOV AX, @DATA 11 MOV DS, AX 12 13 14 LEA DX, msg1 15 MOV AH, 09H 16 INT 21H 17 18 19 MOV AH, 01H 20 INT 21H 21 MOV BL, AL 22 23 24 LEA DX, msg2 25 MOV AH, 09H 26 INT 21H 27 28 29 MOV DL, BL 30 MOV AH, 02H 31 INT 21H 32 33 34 MOV AH, 4CH 35 INT 21H 36 37 MAIN ENDP 38 END MAIN</pre>	 <pre>emulator screen (80x25 chars) Enter a character: j You entered: j</pre>

Conclusion:

From this experiment, we learned how to take input from the user and display output using Assembly Language programming. We also learned the use of registers and DOS interrupt services for performing input and output operations. The program was successfully executed in EMU8086.

Experiment No: 02

Experiment Name: Write an Assembly Language program to perform addition of two numbers.

Introduction:

Assembly Language is a low-level programming language that is used to communicate directly with computer hardware. It helps programmers understand the internal working process of a computer system. In this experiment, an Assembly Language program was developed to perform the addition of two numbers. The program takes two numbers as input from the user, adds those using Assembly instructions, and displays the result on the screen. This experiment helps to understand basic arithmetic operations, register usage, and input-output functions in Assembly Language.

Code	Output												
newopenexamplessavecompileemulatecalc <pre>org 100h 01 02 03 .DATA 04 05 msg1 db "Enter first number: \$" 06 msg2 db 0Dh,0Ah,"Enter second number: \$" 07 msg3 db 0Dh,0Ah,"Result: \$" 08 09 .CODE 10 11 start: 12 13 mov dx, offset msg1 14 mov ah, 09h 15 int 21h 16 17 mov ah, 01h 18 int 21h 19 sub al, 30h 20 mov bl, al 21 22 mov dx, offset msg2 23 mov ah, 09h 24 int 21h 25 26 mov ah, 01h 27 int 21h 28 sub al, 30h 29 add bl, al 30 31 mov dx, offset msg3 32 mov ah, 09h 33 int 21h 34 35 add bl, 30h 36 37 mov dl, bl 38 mov ah, 02h 39 int 21h 40 41 mov ah, 4Ch 42 int 21h</pre>	00h <pre>db "Enter first number: \$" db 0Dh,0Ah,"Enter second number: \$" db 0Dh,0Ah,"Result: \$" : ov dx, offset msg1 ov ah, 09h nt 21h ov ah, 01h nt 21h ub al, 30h ov bl, al ov dx, offset msg2 ov ah, 09h nt 21h ov ah, 01h nt 21h ub al, 30h dd bl, al ov dx, offset msg3 ov ah, 09h nt 21h dd bl, 30h ov dl, bl ov ah, 02h nt 21h ov ah, 4Ch nt 21h</pre> emulator screen (80x25 chars) <pre>Enter first number: 3 Enter second number: 4 Result: 7</pre> emulator: problem 2.com filemathdebugviewexternalvirtual devicesvirtual drivehelp Loadreloadstep backsingle steprunstep delay ms: 0 registers <table><tr><th></th><th>H</th><th>L</th></tr><tr><td>AX</td><td>4C</td><td>37</td></tr><tr><td>BX</td><td>00</td><td>37</td></tr><tr><td>CX</td><td>00</td><td>6C</td></tr></table> F400:0204 message PROGRAM HAS RETURNED CONTROL TO THE OPERATING SYSTEM		H	L	AX	4C	37	BX	00	37	CX	00	6C
	H	L											
AX	4C	37											
BX	00	37											
CX	00	6C											

Conclusion:

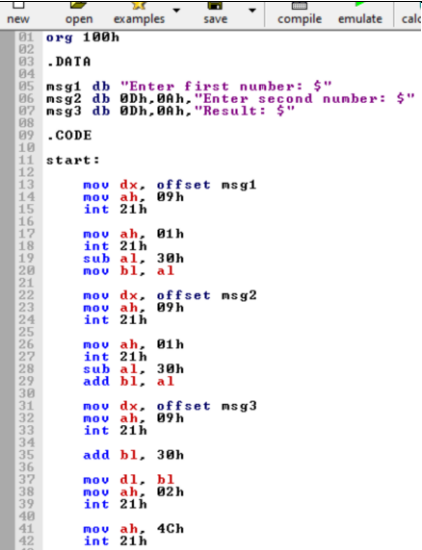
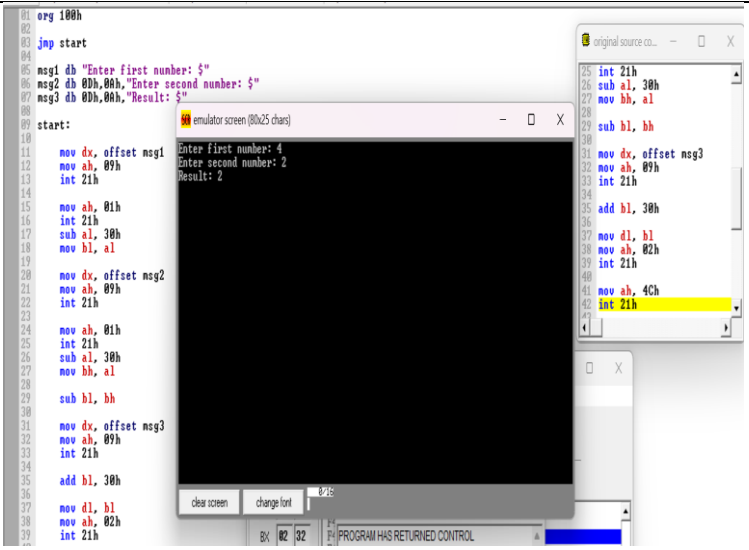
From this experiment, we learned how to perform the addition of two numbers using Assembly Language programming. We also understood the use of registers and arithmetic instructions for processing data. The program was successfully executed in EMU8086 and produced the correct output.

Experiment No: 03

Experiment Name: Write an Assembly Language program to perform subtraction of two numbers.

Introduction:

It is useful for understanding the internal operations of a computer system. In this experiment, an Assembly Language program was written to perform the subtraction of two numbers. The program takes two numbers as input from the user, subtracts one number from another using arithmetic instructions, and displays the result on the screen. This experiment helps to understand arithmetic operations, register manipulation, and input-output techniques in Assembly Language programming.

Code	Output
 <pre>01 org 100h 02 03 .DATA 04 05 msg1 db "Enter first number: \$" 06 msg2 db 0Dh,0Ah,"Enter second number: \$" 07 msg3 db 0Dh,0Ah,"Result: \$" 08 09 .CODE 10 11 start: 12 13 mov dx, offset msg1 14 mov ah, 09h 15 int 21h 16 17 mov ah, 01h 18 int 21h 19 sub al, 30h 20 mov bl, al 21 22 mov dx, offset msg2 23 mov ah, 09h 24 int 21h 25 26 mov ah, 01h 27 int 21h 28 sub al, 30h 29 add bl, al 30 31 mov dx, offset msg3 32 mov ah, 09h 33 int 21h 34 35 add bl, 30h 36 37 mov dl, bl 38 mov ah, 02h 39 int 21h 40 41 mov ah, 4Ch 42 int 21h</pre>	 <pre>01 org 100h 02 03 jmp start 04 05 msg1 db "Enter first number: \$" 06 msg2 db 0Dh,0Ah,"Enter second number: \$" 07 msg3 db 0Dh,0Ah,"Result: \$" 08 09 start: 10 11 mov dx, offset msg1 12 mov ah, 09h 13 int 21h 14 15 mov ah, 01h 16 int 21h 17 sub al, 30h 18 mov bl, al 19 20 mov dx, offset msg2 21 mov ah, 09h 22 int 21h 23 24 mov ah, 01h 25 int 21h 26 sub al, 30h 27 mov bh, al 28 29 sub bl, bh 30 31 mov dx, offset msg3 32 mov ah, 09h 33 int 21h 34 35 add bl, 30h 36 37 mov dl, bl 38 mov ah, 02h 39 int 21h 40 41 mov ah, 4Ch 42 int 21h</pre> emulator screen (80x25 chars) Enter first number: 4 Enter second number: 2 Result: 2 clear screen change font PROGRAM HAS RETURNED CONTROL

Conclusion:

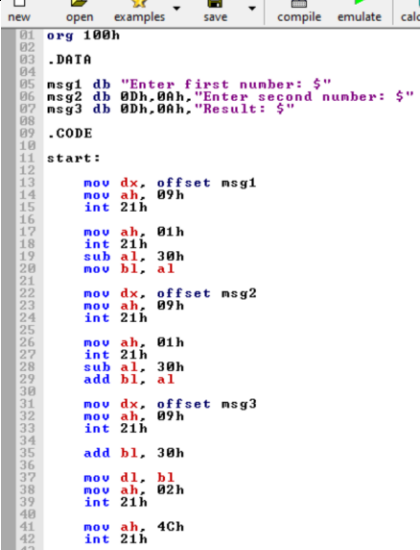
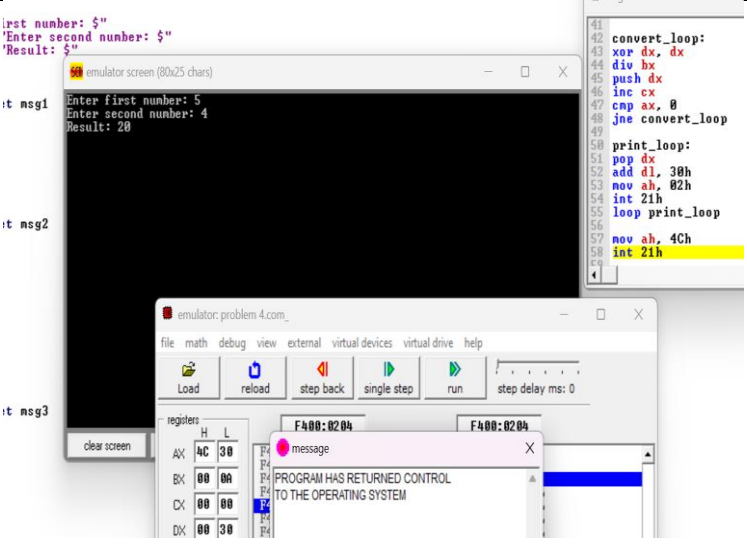
From this experiment, we learned how to perform subtraction between two numbers using Assembly Language. Also gained knowledge about the use of registers, subtraction instructions, and DOS interrupt functions for input and output operations. The program was successfully executed in EMU8086 and produced the expected result.

Experiment No: 04

Experiment Name: Write an Assembly Language program to perform multiplication of two numbers.

Introduction:

It helps programmers understand how arithmetic and logical operations are performed inside the computer. In this experiment, an Assembly Language program was developed to perform the multiplication of two numbers. The program takes two numbers as input from the user, multiplies those using Assembly instructions, and displays the result on the screen. This experiment is important for understanding arithmetic operations, register usage, and data processing in Assembly Language programming.

Code	Output										
 <pre>01 org 100h 02 03 .DATA 04 05 msg1 db "Enter first number: \$" 06 msg2 db 0Dh,0Ah,"Enter second number: \$" 07 msg3 db 0Dh,0Ah,"Result: \$" 08 09 .CODE 10 11 start: 12 13 mov dx, offset msg1 14 mov ah, 09h 15 int 21h 16 17 mov ah, 01h 18 int 21h 19 sub al, 30h 20 mov bl, al 21 22 mov dx, offset msg2 23 mov ah, 09h 24 int 21h 25 26 mov ah, 01h 27 int 21h 28 sub al, 30h 29 add bl, al 30 31 mov dx, offset msg3 32 mov ah, 09h 33 int 21h 34 35 add bl, 30h 36 37 mov dl, bl 38 mov ah, 02h 39 int 21h 40 41 mov ah, 4Ch 42 int 21h</pre>	 <pre>Enter first number: 5 Enter second number: 4 Result: 20</pre> <table border="1"><thead><tr><th>Register</th><th>Value</th></tr></thead><tbody><tr><td>AX</td><td>4C 30</td></tr><tr><td>BX</td><td>00 00</td></tr><tr><td>CX</td><td>00 00</td></tr><tr><td>DX</td><td>00 30</td></tr></tbody></table>	Register	Value	AX	4C 30	BX	00 00	CX	00 00	DX	00 30
Register	Value										
AX	4C 30										
BX	00 00										
CX	00 00										
DX	00 30										

Conclusion:

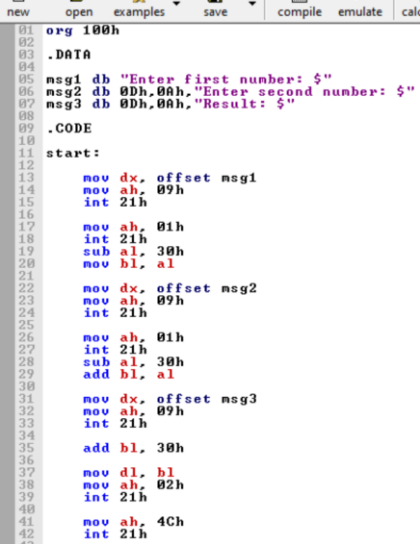
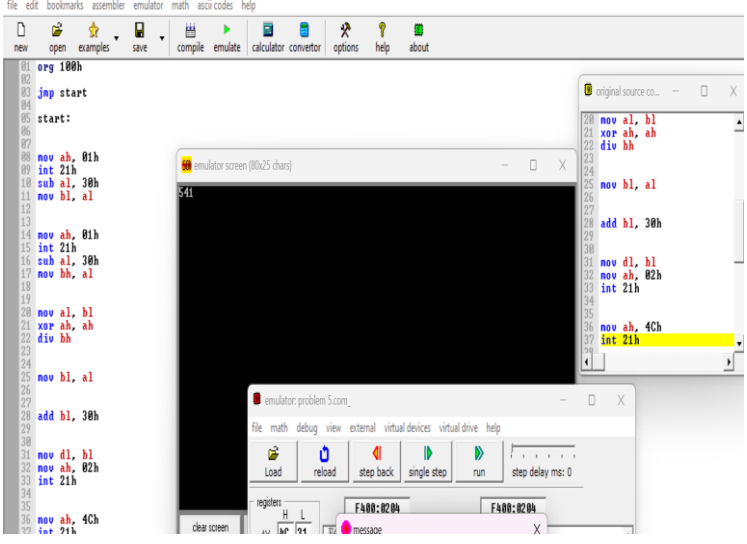
From this experiment, we learned how to perform multiplication of two numbers using Assembly Language. We also understood the use of multiplication instructions, registers, and DOS interrupt functions for input and output operations. The program was successfully executed in EMU8086.

Experiment No: 05

Experiment Name: Write an Assembly Language program to perform division of two numbers.

Introduction:

In this experiment, a program was written to perform the division of two numbers and display the result. This experiment helps to understand division instructions and basic input-output operations in Assembly Language.

Code	Output
 <pre>01 org 100h 02 03 .DATA 04 05 msg1 db "Enter first number: \$" 06 msg2 db 0Dh,0Ah,"Enter second number: \$" 07 msg3 db 0Dh,0Ah,"Result: \$" 08 09 .CODE 10 11 start: 12 13 mov dx, offset msg1 14 mov ah, 09h 15 int 21h 16 17 mov ah, 01h 18 int 21h 19 sub al, 30h 20 mov bl, al 21 22 mov dx, offset msg2 23 mov ah, 09h 24 int 21h 25 26 mov ah, 01h 27 int 21h 28 sub al, 30h 29 add bl, al 30 31 mov dx, offset msg3 32 mov ah, 09h 33 int 21h 34 35 add bl, 30h 36 37 mov dl, bl 38 mov ah, 02h 39 int 21h 40 41 mov ah, 4Ch 42 int 21h</pre>	 The screenshot shows the EMU8086 emulator interface. The assembly code from the previous block is loaded and running. The 'emulator screen (80x25 chars)' window displays the output of the program, which consists of three prompts and their corresponding inputs and results. The 'registers' window shows the state of the CPU registers, with the EAX register containing the result of the division (020A).

Conclusion:

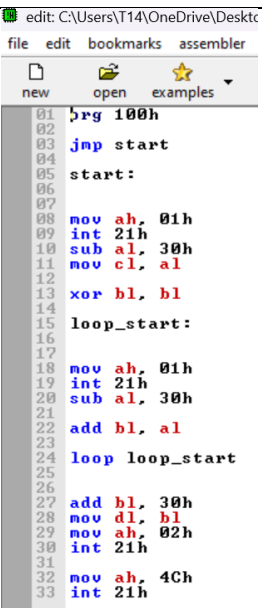
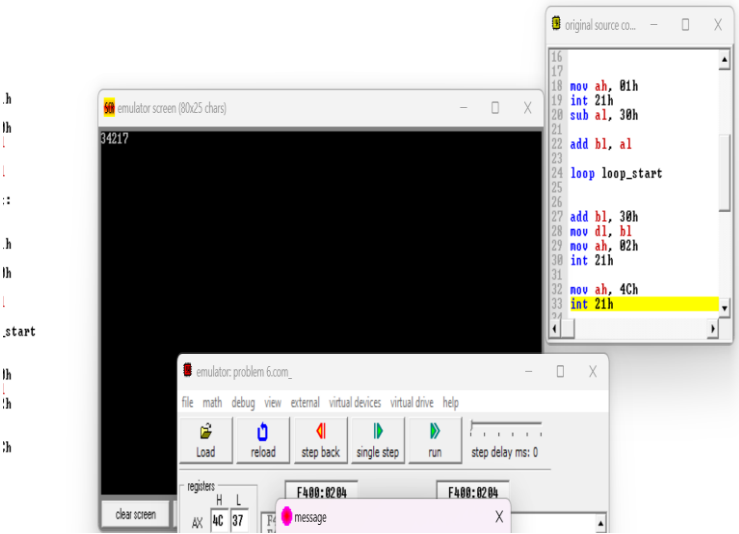
From this experiment, we learned how to divide two numbers using Assembly Language and display the output correctly. The program was successfully executed in EMU8086 and helped us understand basic arithmetic operations and register usage.

Experiment No: 06

Experiment Name: Write an Assembly Language program to take input from the user and display the output.

Introduction:

In this experiment, a program was developed to calculate the sum of N numbers using a loop. The program repeatedly takes input values, adds them using a loop, and displays the final result. This experiment helps to understand loop control, register usage, and arithmetic operations in Assembly Language.

Code	Output
 <pre>edit: C:\Users\T14\OneDrive\Desktop file edit bookmarks assembler new open examples 01 org 100h 02 03 jmp start 04 05 start: 06 07 08 mov ah, 01h 09 int 21h 10 sub al, 30h 11 mov cl, al 12 13 xor bl, bl 14 15 loop_start: 16 17 18 mov ah, 01h 19 int 21h 20 sub al, 30h 21 22 add bl, al 23 24 loop loop_start 25 26 27 add bl, 30h 28 mov dl, bl 29 mov ah, 02h 30 int 21h 31 32 mov ah, 4Ch 33 int 21h</pre>	 The output window shows the result of the program execution, which is 34217. The registers window shows the values of the registers: AX: 4C, CX: 37, DX: 02, and the message box displays the sum 34217.

Conclusion:

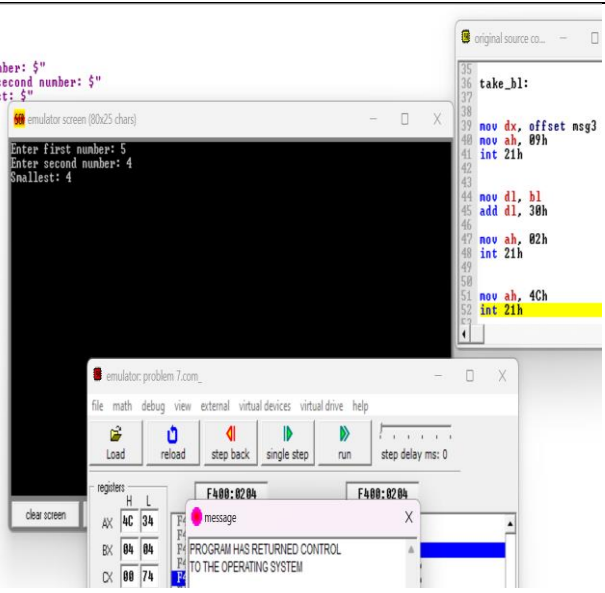
From this experiment, we learned how to calculate the sum of N numbers using a loop in Assembly Language. We also understood how loops and registers are used to perform repeated addition. The program was successfully executed in EMU8086 and produced the correct result, improving our understanding of iterative operations in Assembly Language.

Experiment No: 07

Experiment Name: Write an Assembly Language program to find the smallest number between two given numbers.

Introduction:

In this experiment, a program was written to find the smallest number between two given numbers. The program compares two input values and displays the smaller one as the result. This experiment helps to understand comparison operations and basic decision-making in Assembly Language.

Code	Output
<pre>01 org 100h 02 03 jmp start 04 05 msg1 db "Enter first number: \$" 06 msg2 db 0Dh,0Ah,"Enter second number: \$" 07 msg3 db 0Dh,0Ah,"Smallest: \$" 08 09 start: 10 11 mov dx, offset msg1 12 mov ah, 09h 13 int 21h 14 15 mov ah, 01h 16 int 21h 17 sub al, 30h 18 mov bl, al 19 20 mov dx, offset msg2 21 mov ah, 09h 22 int 21h 23 24 mov ah, 01h 25 int 21h 26 sub al, 30h 27 mov bh, al 28 29 cmp bl, bh 30 jbe take_b1 31 mov bl, bh 32 33 take_b1: 34 35 mov dx, offset msg3 36 mov ah, 09h 37 int 21h 38 39 mov dl, bl 40 add dl, 30h 41 42 43 44 45 46</pre>	<pre>rg 100h np start sg1 db "Enter first number: \$" sg2 db 0Dh,0Ah,"Enter second number: \$" sg3 db 0Dh,0Ah,"Smallest: \$" tart: mov dx, offset msg1 mov ah, 09h int 21h mov ah, 01h int 21h sub al, 30h mov bl, al mov dx, offset msg2 mov ah, 09h int 21h mov ah, 01h int 21h sub al, 30h mov bh, al cmp bl, bh jbe take_b1 mov bl, bh ake_b1: mov dx, offset msg3 mov ah, 09h int 21h mov dx, offset msg3 mov ah, 09h int 21h</pre> 

Conclusion:

From this experiment, we learned how to find the smallest number using Assembly Language. We also understood how comparison instructions work to make decisions between values. The program was successfully executed in EMU8086 and produced the correct output.

Experiment Name: Write an Assembly Language program to find the smallest number among three given numbers.

Introduction: In this experiment, a program was written to find the smallest number among three given numbers. The program compares the input values and identifies the smallest one. This experiment helps to understand comparison operations and decision-making in Assembly Language.

Code	Output
<pre> 01 org 100h 02 03 jmp start 04 05 start: 06 07 08 mov ah, 01h 09 int 21h 10 sub al, 30h 11 mov bl, al 12 13 mov ah, 01h 14 int 21h 15 sub al, 30h 16 mov bh, al 17 18 19 mov ah, 01h 20 int 21h 21 sub al, 30h 22 mov cl, al 23 24 25 mov al, bl 26 27 cmp bh, al 28 jge skip1 29 mov al, bh 30 31 skip1: 32 cmp cl, al 33 jge skip2 34 mov al, cl 35 36 skip2: 37 38 39 xor ah, ah 40 41 add al, 30h 42 43 mov dl, al 44 mov ah, 02h 45 int 21h 46 </pre>	

Conclusion:

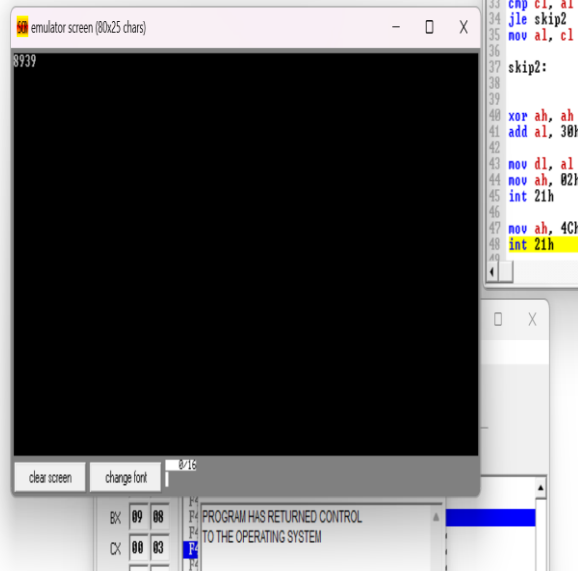
From this experiment, we learned how to find the smallest number among three values using Assembly Language. We also understood how multiple comparisons are performed using conditional logic. The program was successfully executed in EMU8086 and produced the correct result.

Experiment No: 09

Experiment Name: Write an Assembly Language program to find the largest number among three given numbers.

Introduction:

In this experiment, a program was written to find the largest number among three given numbers. The program compares the values and selects the greatest one. This helps to understand comparison and decision-making in Assembly Language.

Code	Output
<pre>01 org 100h 02 03 jmp start 04 05 start: 06 07 08 mov ah, 01h 09 int 21h 10 sub al, 30h 11 mov bl, al 12 13 14 mov ah, 01h 15 int 21h 16 sub al, 30h 17 mov bh, al 18 19 20 mov ah, 01h 21 int 21h 22 sub al, 30h 23 mov cl, al 24 25 26 mov al, bl 27 28 cmp bh, al 29 jle skip1 30 mov al, bh 31 32 skip1: 33 cmp cl, al 34 jle skip2 35 mov al, cl 36 37 skip2: 38 39 xor ah, ah 40 add al, 30h 41 42 43 mov dl, al 44 mov ah, 02h 45 int 21h 46</pre>	

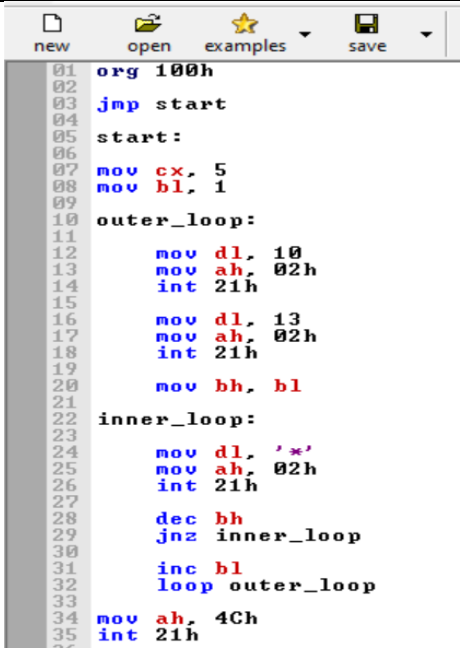
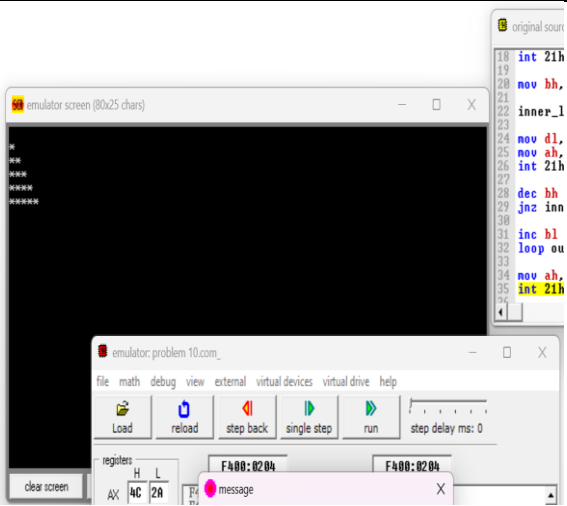
Conclusion: From this experiment, we learned how to find the largest number among three values using Assembly Language. The program was successfully executed in EMU8086 and helped us understand basic comparison operations.

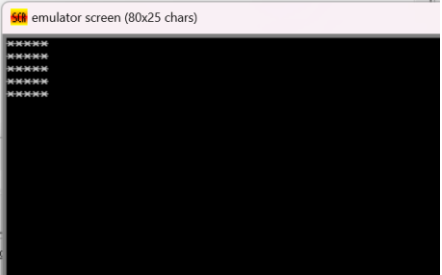
Experiment No: 10

Experiment Name: Write an Assembly Language program to print a simple pattern (e.g., triangle, square, or pyramid).

Introduction:

In this experiment, a program was written to print simple patterns like triangle, square, or pyramid. This helps to understand loop usage and output handling in Assembly Language.

Code	Output
 <pre>01 org 100h 02 jmp start 03 start: 04 05 06 07 mov cx, 5 08 mov bl, 1 09 10 outer_loop: 11 12 mov dl, 10 13 mov ah, 02h 14 int 21h 15 16 mov dl, 13 17 mov ah, 02h 18 int 21h 19 20 mov bh, bl 21 22 inner_loop: 23 24 mov dl, '*' 25 mov ah, 02h 26 int 21h 27 28 dec bh 29 jnz inner_loop 30 31 inc bl 32 loop outer_loop 33 34 mov ah, 4Ch 35 int 21h</pre>	 <pre>p: l, 10 h, 02h lh lh l, 13 h, 02h lh h, bl p: l, '*' h, 02h lh h inner_loop l outer_loop Ch</pre>

Code(Square)	Output
<pre> 01 org 100h 02 03 jmp start 04 05 start: 06 07 mov cx, 5 08 09 outer: 10 11 mov bx, 5 12 13 inner: 14 15 mov dl, '*' 16 mov ah, 02h 17 int 21h 18 19 dec bx 20 jnz inner 21 22 mov dl, 10 23 mov ah, 02h 24 int 21h 25 26 mov dl, 13 27 mov ah, 02h 28 int 21h 29 loop outer 30 31 mov ah, 4Ch 32 int 21h 33 34 </pre>	<pre> p start art: v cx, 5 ter: mov bx, 5 ner: mov dl, '*' mov ah, 02h int 21h dec bx jnz inner mov dl, 10 mov ah, 02h int 21h mov dl, 13 mov ah, 02h int 21h </pre> 

Code(pyramid)	Output
<pre> 01 org 100h 02 03 jmp start 04 05 start: 06 07 mov cx, 5 08 mov bl, 1 09 10 outer: 11 12 push cx 13 14 mov al, 5 15 sub al, bl 16 mov cl, al 17 18 space_loop: 19 cmp cl, 0 20 je star_loop 21 mov dl, ' ' 22 mov ah, 02h 23 int 21h 24 dec cl 25 jmp space_loop 26 27 star_loop: 28 mov bh, bl 29 30 print_star: 31 cmp bh, 0 32 je newline 33 mov dl, '*' 34 mov ah, 02h 35 int 21h 36 dec bh 37 jmp print_star 38 39 newline: 40 41 42 43 44 45 46 </pre>	

Conclusion:

From this experiment, we learned how to print simple patterns using loops in Assembly Language. The program was successfully executed in EMU8086 and improved our understanding of iterative programming.